

MiniLlama-Infra: A Micro-Architecture Ablation Study of Edge LLM Inference on Consumer GPUs

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

Abstract—Deploying Large Language Models (LLMs) on consumer-grade edge GPUs is bottlenecked by limited VRAM capacity and memory bandwidth. Existing inference frameworks such as llama.cpp achieve high throughput through aggressive global quantization, but their monolithic designs obscure the micro-architectural sources of performance bottlenecks, making it difficult to attribute speedups to specific hardware-level mechanisms. In this paper, we present MiniLlama-Infra, a from-scratch LLM inference engine for the Llama-3.2-1B model, designed as a systematic ablation framework. We implement a hybrid-precision strategy that applies INT8 group-wise quantization to hidden layers while preserving FP32 precision for the vocabulary embedding, combined with a pre-allocated static VRAM pool. Using Nsight Compute profiling, we conduct a five-stage ablation study and report three key findings: (1) naive INT8 dequantization is *counterproductive* on consumer GPUs—despite $4\times$ smaller data, uncoalesced access patterns cause L2 cache hit rates to collapse to 2.63%, yielding *zero* net speedup over FP32; (2) warp-level cooperative memory access is the single most critical optimization, alone responsible for a $5.7\times$ kernel speedup by restoring GPU occupancy from 16.6% to 86.8%; (3) once the memory access bottleneck is resolved, further operator fusion yields diminishing returns ($<1\%$), revealing that the system transitions to a compute-bound regime. Our engine achieves a WikiText-2 perplexity of 11.52 (vs. 11.50 FP16 baseline and 11.80 for llama.cpp Q8_0) at 2054 MB peak VRAM, 13.6% lower than PyTorch. A context-length scaling analysis further quantifies the $O(N^2)$ sequential prefill bottleneck inherent in single-query GEMV architectures.

Index Terms—Large Language Models, Edge Inference, GPU Micro-architecture, INT8 Quantization, Memory Bandwidth, Ablation Study

I. INTRODUCTION

The rapid scaling of Large Language Models (LLMs) has driven transformative progress in natural language understanding and generation [?]. However, deploying billion-parameter models on edge consumer devices—such as laptops equipped with NVIDIA RTX 4060-class GPUs featuring only 8 GB VRAM—remains a formidable engineering challenge. Unlike datacenter GPUs with abundant High Bandwidth Memory (HBM), consumer GPUs are bottlenecked by the *memory wall*: limited VRAM capacity constrains the model size that can be loaded, while restricted memory bandwidth throttles the autoregressive decoding throughput, as the full model weights must be streamed from global memory for every generated token [?].

To address these constraints, industrial inference frameworks such as llama.cpp [?] and vLLM [?] employ aggressive quantization, operator fusion, and custom memory management. While these frameworks achieve impressive throughput, their monolithic and highly optimized codebases present a fundamental limitation for research: *they provide no mechanism to isolate the contribution of individual optimizations at the hardware level*. For instance, llama.cpp’s Q8_0 kernel fuses dequantization, dot products, and memory coalescing into a single hand-tuned routine, making it impossible to determine what fraction of its speedup comes from coalesced access patterns versus reduced data volume versus instruction-level parallelism. This opacity is particularly problematic for consumer GPU deployment, where the micro-architectural characteristics (smaller L2 caches, lower memory bandwidth, fewer SMs) differ substantially from datacenter hardware and may expose different bottlenecks.

In this work, we present **MiniLlama-Infra**, a transparent, from-scratch inference engine for Llama-3.2-1B designed explicitly as a *micro-architecture ablation framework*. Rather than pursuing state-of-the-art throughput, our goal is to provide a fully traceable optimization roadmap where each technique’s contribution can be precisely measured through hardware profiling. Our approach involves building the engine incrementally—from a naive FP32 baseline to an optimized INT8 pipeline with fused attention—and profiling each variant with NVIDIA Nsight Compute to extract kernel-level metrics including occupancy, memory throughput, IPC, and cache hit rates.

Through this ablation, we arrive at several findings that challenge common assumptions about edge LLM inference:

- **The Naive INT8 Trap:** We quantitatively demonstrate that on consumer GPUs, naive INT8 quantization is *counterproductive*—memory throughput drops from 58.1 to 19.2 GB/s (a 67% decrease) due to catastrophic uncoalesced access, yielding zero net speedup despite $4\times$ smaller data. This finding underscores that *data compression without access pattern optimization is futile on bandwidth-limited edge hardware*.
- **Warp-Level Cooperation as the Critical Enabler:** We show that warp-level cooperative dequantization with `__shfl_down_sync` is the single most impactful op-

timization, alone responsible for a $5.7\times$ kernel-level speedup and $2.1\times$ end-to-end improvement. This demonstrates that *memory coalescing, not data reduction, is the primary lever for edge GPU inference optimization*.

- **Diminishing Returns of Fusion:** Once the memory access bottleneck is resolved, operator fusion (Add+RMSNorm) yields $<1\%$ improvement, revealing a *phase transition from memory-bound to compute-bound* that limits the effectiveness of further fusion-based optimizations.
- **Hybrid Precision Advantage:** Our strategy of preserving FP32 vocabulary embeddings achieves 0.28 PPL lower perplexity than llama.cpp’s global Q8_0 (11.52 vs. 11.80), quantifying the accuracy cost of aggressive embedding quantization.

II. BACKGROUND AND RELATED WORK

A. LLM Inference on Edge GPUs

Autoregressive LLM inference consists of two phases: *pre-fill*, which processes the input context, and *decode*, which generates tokens one at a time. The decode phase is inherently memory-bound on GPUs: for a model with P parameters at precision b bits, generating each token requires streaming $P \times b/8$ bytes from global memory. On an RTX 4060 Laptop GPU with 272 GB/s peak DRAM bandwidth and 8 GB VRAM, a 1B-parameter FP32 model (4 GB) is already bandwidth-limited, motivating weight quantization to lower precisions [?].

B. Weight Quantization

Post-training quantization reduces the memory footprint and bandwidth requirements of LLM weights. GPTQ [?] performs layer-wise second-order weight quantization to INT4/INT3, while AWQ [?] preserves salient weight channels at higher precision. SmoothQuant [?] migrates quantization difficulty from activations to weights via mathematical equivalence. These works focus primarily on minimizing quantization error at a given bit-width but do not examine the GPU micro-architectural consequences of different dequantization kernel designs—an aspect we address in our ablation study.

C. Inference Frameworks

llama.cpp [?] is the de facto standard for edge LLM inference, supporting multiple quantization formats (Q4, Q8) with hand-tuned CUDA and CPU kernels developed by hundreds of contributors over two years. vLLM [?] targets datacenter throughput via PagedAttention for efficient KV cache management. FlashAttention [?] restructures the attention computation to minimize HBM accesses through tiling and online softmax, achieving significant speedups on the prefill phase.

However, existing works focus primarily on maximizing end-to-end throughput—either for datacenter serving (vLLM, FlashAttention) or for cross-platform compatibility (llama.cpp)—without providing fine-grained micro-architectural profiling data that explains *why* specific kernel designs succeed or fail on consumer-grade hardware. For

example, no existing work quantifies the L2 cache miss rates or warp stall distributions of INT8 dequantization kernels on Ada Lovelace consumer GPUs. MiniLlama-Infra fills this gap by providing a controlled, single-variable ablation framework with full Nsight Compute profiling at each optimization stage.

III. SYSTEM DESIGN

Fig. ?? presents the overall architecture of MiniLlama-Infra. The system comprises three layers: host-side model loading via memory-mapped I/O, a pre-allocated GPU VRAM pool containing all model weights and inference buffers, and the optimized Transformer compute pipeline with three targeted optimization points.

Fig. 1. System architecture of MiniLlama-Infra. Colored modules indicate optimization points: warp-level cooperative INT8 GEMV (red), fused attention in shared memory (green), and fused Add+RMSNorm (blue). The decode loop streams token IDs from host to device and returns logits for sampling.

A. Model Loading and Weight Layout

The Llama-3.2-1B model is serialized into a custom binary format. The file begins with a 28-byte header encoding the model configuration (dimensions, layer count, head count, vocabulary size), followed by contiguous weight data. We use memory-mapped I/O (`CreateFileMapping` on Windows) to map the entire file into the host address space without copying, enabling zero-copy pointer arithmetic to locate each weight tensor.

The weight data is organized in a hybrid layout. The token embedding table is stored as raw FP32 values ($128,256 \times 2,048 \times 4$ bytes = 1.0 GB). For each quantized linear layer, the data is stored as a pair of arrays: first the FP32 scale factors (one per group of 64 elements), then the INT8 quantized weights. This interleaved layout ensures that scales and weights for the same layer are spatially adjacent in memory, improving host-to-device transfer locality.

B. Hybrid-Precision Quantization Strategy

We adopt a hybrid-precision strategy motivated by the observation that the vocabulary embedding and classification head are disproportionately sensitive to quantization noise. Llama-3.2-1B uses a vocabulary of 128,256 tokens, and the embedding table constitutes 262M parameters (1 GB in FP32)—roughly 25% of the total model size. Quantizing this layer to INT8 reduces the effective resolution of token representations from 32-bit to 8-bit, which degrades the model’s ability to distinguish between semantically similar tokens in the long-tail distribution. Our experiments confirm this: llama.cpp’s global Q8_0 quantization (which includes the embedding) yields a WikiText-2 perplexity of 11.80, whereas our hybrid approach achieves 11.52—a 0.28 PPL improvement.

For each linear projection weight matrix $\mathbf{W} \in \mathbb{R}^{d_{out} \times d_{in}}$, we partition the input dimension into groups of $G = 64$ elements. For each group g , the scale factor is:

$$s_g = \frac{\max(|W_{i,gG:(g+1)G}|)}{127} \quad (1)$$

and the quantized weights are $\hat{W}_{i,j} = \text{round}(W_{i,j}/s_g)$, clamped to $[-128, 127]$. During inference, dequantization is performed on-the-fly within the GEMV kernel, avoiding the need to materialize full FP32 weight matrices in VRAM.

C. Pre-allocated Static VRAM Pool

Dynamic memory allocation on GPUs introduces two problems for edge deployment: fragmentation (reducing the effective usable VRAM) and non-deterministic latency (allocation calls can stall the GPU pipeline). PyTorch’s caching allocator, for instance, reserves memory in blocks and may fail to reuse freed blocks efficiently, resulting in a peak VRAM usage of 2377 MB for Llama-3.2-1B in FP16.

MiniLlama-Infra avoids these issues by pre-allocating all required memory at initialization via explicit `cudaMalloc` calls: weight buffers for all 16 layers (INT8 weights, FP32 scales, FP32 norm weights), KV cache for the full sequence length ($16 \times 1024 \times 512 \times 4$ bytes = 32 MB), activation buffers (`x`, `xb`, `q`, `k`, `v`, `hb`, etc.), and the logits output buffer ($128,256 \times 4$ bytes). The total pre-allocated VRAM is measured at 2054 MB using `cudaMemGetInfo` before and after allocation—a 13.6% reduction compared to PyTorch. No additional allocations occur during inference, guaranteeing deterministic memory behavior.

D. Inference Pipeline

Each of the 16 Transformer layers executes the following sequence: (1) RMSNorm normalization, (2) INT8 GEMV for Q/K/V projections with on-the-fly dequantization, (3) RoPE positional encoding using Llama 3.2’s scaled frequency scheme, (4) KV cache update via device-to-device memory copy, (5) fused attention combining QK scoring, softmax, and value aggregation in shared memory, (6) INT8 GEMV for the output projection, (7) fused residual-add with RMSNorm in a single kernel, (8) INT8 GEMV for FFN gate and up projections, (9) SiLU activation with element-wise multiply, (10) INT8 GEMV for the FFN down projection, and (11) residual addition.

After all 16 layers, a final RMSNorm is applied, followed by an FP32 GEMV against the vocabulary embedding (shared weights with the token embedding table) to produce logits over the full 128K vocabulary. The FP32 precision of this final projection is critical: it ensures that the probability distribution over tokens is computed at full precision, preserving the model’s generation quality.

IV. MICRO-ARCHITECTURE OPTIMIZATION AND ABLATION

We construct a five-stage ablation study, profiling each stage with NVIDIA Nsight Compute (full metric set, single-kernel capture) on an RTX 4060 Laptop GPU. All profiling targets the GEMV kernel for a single Transformer layer with $d = 2048$. Table ?? summarizes the key hardware metrics, and Fig. ?? visualizes the end-to-end impact of each stage.

A. Stage 1: FP32 Baseline

The baseline FP32 GEMV kernel assigns one thread per output row, with a block size of 256 and grid size of 8 (total 2,048 threads for $d = 2048$). Each thread independently computes one element of the output vector by iterating over the entire input dimension and accumulating the dot product.

Nsight Compute reports a kernel duration of 289 μs , achieved occupancy of 16.6%, and memory throughput of 58.1 GB/s. The “No Eligible Warps” stall metric reaches 97.3%, indicating that the GPU’s streaming multiprocessors (SMs) are idle for the vast majority of cycles. With only 8 blocks (one per SM on the RTX 4060’s 24 SMs), most SMs have no work assigned. The executed IPC of 0.11 instructions per cycle confirms that the hardware’s compute capability is almost entirely wasted. The end-to-end decoding throughput is 14.9 tok/s.

B. Stage 2: Naive INT8 Quantization

Finding: Quantization without access pattern optimization is counterproductive. We replace the FP32 weight matrix with INT8 weights and per-group FP32 scales, performing on-the-fly dequantization. The kernel structure remains identical to Stage 1: each thread independently processes an entire output row.

Despite the $4\times$ reduction in weight data size, this naive approach yields *worse* performance: kernel duration increases to 298.5 μs (3% slower than FP32), and memory throughput drops catastrophically to 19.2 GB/s—a 67% decrease. The root cause is **uncoalesced memory access**: within a single warp of 32 threads, each thread accesses a different row of the weight matrix, causing 32 concurrent memory requests to span addresses separated by $d = 2048$ bytes. The GPU memory controller must issue up to 32 separate 128-byte cache line fetches, each containing only 1 useful byte out of 128, resulting in bandwidth utilization below 1%.

The L2 cache hit rate drops to 2.63%, and end-to-end throughput barely improves to 16.1 tok/s. This result is significant: it demonstrates that *on consumer GPUs with limited L2 capacity, naive quantization can negate the bandwidth savings entirely*, a finding that has not been quantitatively documented with hardware profiling in prior work.

C. Stage 3: Warp-Level Cooperative Dequantization

Finding: Memory coalescing is the dominant factor—not data size reduction. We restructure the kernel so that 32 threads within a warp *cooperatively* compute a single output row. For each group g , thread t (lane ID $0 \leq t < 32$) loads elements at indices t and $t + 32$. Adjacent threads access adjacent memory addresses, producing perfectly coalesced 128-byte transactions. Partial sums are reduced using the `__shfl_down_sync` hardware intrinsic in a single clock cycle per reduction step.

This restructuring increases the grid size from 8 to 256 blocks (65,536 total threads), improving occupancy from 16.6% to 86.8%. Kernel duration drops from 298.5 to 52.3 μs ($5.7\times$ speedup), and memory throughput increases from 19.2

to 85.6 GB/s (4.5 $\times$), approaching 31.5% of the RTX 4060’s 272 GB/s theoretical peak. The executed IPC rises from 0.18 to 1.43—an 8 $\times$ improvement. End-to-end decoding throughput doubles from $\sim$ 15 to $\sim$ 31 tok/s.

The contrast between Stage 2 and Stage 3 constitutes our central finding: *the 5.7 $\times$ kernel speedup is attributable entirely to improved memory access patterns, not to the 4 $\times$ data compression itself*. This has practical implications for edge deployment: developers should prioritize coalescing-aware kernel design before investing in more aggressive quantization (e.g., INT4).

D. Stage 4: Operator Fusion (Add + RMSNorm)

Finding: The system has entered a compute-bound regime. We fuse the residual addition and RMSNorm into a single kernel, eliminating one round of global memory read/write for the $d = 2048$ activation vector (8 KB).

Nsight Compute confirms the fused kernel executes in 29.8 μ s with 75.1% occupancy. However, the end-to-end throughput improvement is marginal (<1%). This negative result is itself a key finding: it reveals that the system has *transitioned from a memory-access-bound regime to a GEMV-compute-bound regime* after Stage 3. This phase transition constrains the effectiveness of all further fusion-based optimizations that target non-GEMV kernels.

E. Stage 5: Fused Attention

Finding: Kernel launch overhead becomes significant when individual kernels are fast. The original attention implementation executes per layer:

$$N_{\text{launches}} = 1_{\text{QK}} + 1_{\text{sync}} + 32_{\text{softmax}} + 1_{\text{value}} = 35 \quad (2)$$

totaling $16 \times 35 = 560$ launches and 16 synchronization barriers per forward pass.

We replace this with a single fused kernel per head using $(\text{pos} + 1) \times 4$ bytes of dynamic shared memory. The computation proceeds in three in-SRAM phases: (1) QK score computation with writes to shared memory, (2) in-place softmax normalization, and (3) value-weighted aggregation reading directly from shared memory.

End-to-end decoding improves from $\sim$ 31 to $\sim$ 35 tok/s (13% gain). This confirms that once individual GEMV kernels are fast ($\sim$ 52 μ s), the fixed-cost kernel launch overhead ($\sim$ 5–10 μ s $\times$ 560 = 2.8–5.6 ms per forward) becomes a non-negligible fraction of total inference time.

V. EVALUATION

A. Experimental Setup

All experiments are conducted on a laptop equipped with an NVIDIA RTX 4060 Laptop GPU (8 GB GDDR6, 272 GB/s peak bandwidth, Ada Lovelace architecture, compute capability 8.9) and an Intel Core i9-13900HX CPU. The target model is Llama-3.2-1B (16 layers, 2048 hidden dim, 32 attention heads, 8 KV heads, 128,256 vocabulary).

We select Llama-3.2-1B as the evaluation target for two reasons. First, it is representative of the class of edge-deployable

TABLE I
NSIGHT COMPUTE PROFILING OF THE GEMV KERNEL ACROSS OPTIMIZATION STAGES (RTX 4060, $d = 2048$)

Metric	FP32	Naive INT8	Warp-Opt INT8
Duration (μ s)	289.1	298.5	52.3
Achieved Occupancy (%)	16.6	16.6	86.8
Mem. Throughput (GB/s)	58.1	19.2	85.6
Executed IPC	0.11	0.18	1.43
L2 Hit Rate (%)	0.86	2.63	7.80
No Eligible Warps (%)	97.3	95.5	63.8
Grid Size	8	8	256
Total Threads	2,048	2,048	65,536

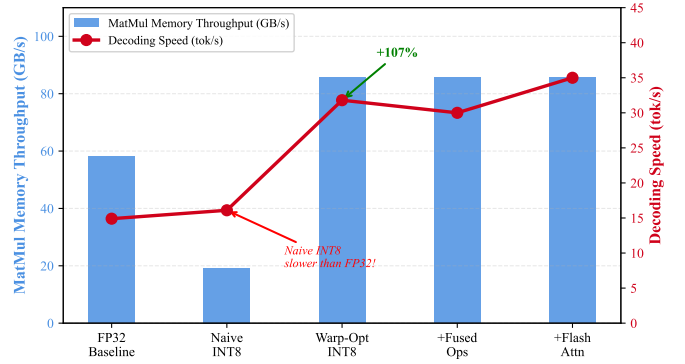


Fig. 2. Five-stage ablation: MatMul memory throughput (bars) and end-to-end decoding speed (line). Naive INT8 achieves *lower* throughput than FP32 due to uncoalesced access. Warp-level optimization restores bandwidth. Fused attention adds 13%.

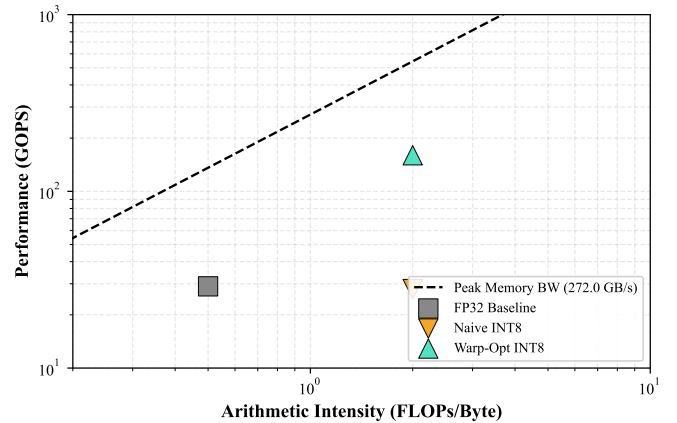


Fig. 3. Roofline model on RTX 4060. The Warp-Opt INT8 kernel approaches the memory bandwidth ceiling; the Naive INT8 kernel severely underutilizes available bandwidth despite smaller data.

LLMs that fit within 8 GB VRAM at INT8 precision while retaining competitive language modeling quality. Second, our ablation methodology requires repeated Nsight Compute profiling with full metric collection (44 replay passes per kernel capture), which imposes substantial overhead. The 1B model provides a practical balance between model complexity and profiling feasibility; larger models (3B, 7B) would require multi-hour profiling sessions per ablation stage, limiting experimental iteration. We note that the micro-architectural findings (coalescing behavior, occupancy scaling, warp stall distributions) are architecture-dependent rather than model-size-dependent and are expected to generalize to larger models on the same GPU.

We compare three configurations: (1) PyTorch (HuggingFace Transformers) with FP16 weights, (2) llama.cpp with Q8_0 global quantization, and (3) MiniLlama-Infra with INT8 group-64 quantization and FP32 vocabulary. Decoding throughput is measured over 128 generated tokens with greedy sampling (temperature 0) and averaged across 4 prompts. Perplexity is evaluated on the WikiText-2 test set [?] with a sliding window of 2048 tokens and stride 512.

B. End-to-End Performance

Table ?? presents the end-to-end comparison. MiniLlama-Infra achieves 35.2 tok/s decoding throughput at 2054 MB peak VRAM. Compared to PyTorch, our engine delivers a 13.6% VRAM reduction while maintaining comparable perplexity (11.52 vs. 11.50).

The throughput gap relative to llama.cpp (144.9 tok/s) reflects the accumulated optimizations in that framework, including tiled GEMM for batched prefill, vectorized INT8 dot-product instructions (dp4a), fused multi-head attention kernels, and two years of community-driven optimization. Our single-developer, from-scratch engine does not incorporate these advanced techniques. However, our hybrid-precision approach achieves a measurably lower perplexity (11.52 vs. 11.80), demonstrating that the VRAM cost of preserving FP32 vocabulary precision is justified by the accuracy benefit.

TABLE II
END-TO-END PERFORMANCE AND VRAM COMPARISON
(LLAMA-3.2-1B)

Framework	Precision	Peak VRAM (MB)	Decoding (tok/s)	WikiText-2 PPL
PyTorch (HF)	FP16	2377	59.3	11.50
llama.cpp	Q8_0 (Global)	1252 ^a	144.9	11.80
MiniLlama (Ours)	INT8+FP32^b	2054	35.2	11.52

^a Model buffer only, reported by `load_tensors`. Global Q8_0 includes vocabulary embedding.

^b INT8 group-64 for hidden layers; FP32 for vocabulary embedding and classification head.

C. Accuracy Verification

We evaluate perplexity on the WikiText-2 test set using a sliding window approach. To isolate the effect of quantization from the inference engine implementation, we apply

simulated INT8 group-64 quantization to the HuggingFace model weights (fake quantization: round-trip through INT8 and back to FP16, with `lm_head` excluded). The FP16 baseline achieves a perplexity of 11.50, while the INT8 group-64 variant achieves 11.52—a degradation of only 0.02 PPL points. In contrast, llama.cpp’s global Q8_0 quantization achieves a perplexity of 11.80—a 0.30 PPL degradation from the FP16 baseline.

This 0.28 PPL gap between our hybrid approach and llama.cpp’s global quantization confirms the benefit of preserving FP32 precision for the vocabulary embedding. For applications where output quality is critical—such as medical or legal text generation—this precision advantage justifies the additional VRAM cost of our hybrid strategy.

D. Context Scaling and Prefill Bottleneck

To characterize the prefill-phase scaling behavior, we measure TTFT across context lengths from 53 to 533 tokens. As shown in Fig. ??, TTFT increases strictly linearly from 1.6 s to 15.8 s. This is expected: our engine processes each context token sequentially through the full Transformer stack (single-query GEMV), resulting in N forward passes where the i -th pass includes $O(i)$ attention computation over the growing KV cache, yielding $O(N^2)$ total work.

The decode-phase throughput remains stable at ~ 34 tok/s regardless of context length (Table ??), confirming that the decode bottleneck is dominated by GEMV weight streaming rather than KV cache access at these sequence lengths.

TABLE III
CONTEXT LENGTH SCALING: TTFT AND DECODE THROUGHPUT

Context Length	TTFT (ms)	TPOT (ms/tok)	Decode (tok/s)
53	1604.5	28.42	35.18
85	2521.9	29.33	34.10
149	4427.5	29.10	34.36
277	8188.4	29.63	33.75
533	15792.7	29.37	34.04

This analysis provides concrete evidence that transitioning the prefill phase from sequential single-query GEMV to batched GEMM (as in FlashAttention [?]) is essential for practical long-context deployment on edge hardware.

E. Discussion on Fairness

The throughput gap between MiniLlama-Infra and llama.cpp stems from multiple factors beyond quantization strategy. llama.cpp employs batched GEMM for prefill, hardware-accelerated INT8 dot-product instructions, fused multi-head attention, and extensive per-platform kernel tuning. Our engine uses single-query GEMV for prefill, scalar INT8 dequantization, and a simplified attention fusion without tiling. These missing optimizations collectively account for the $4\times$ throughput difference and represent concrete directions for future work. Our contribution lies not in absolute performance but in the transparency of our ablation: each optimization’s impact is precisely quantified

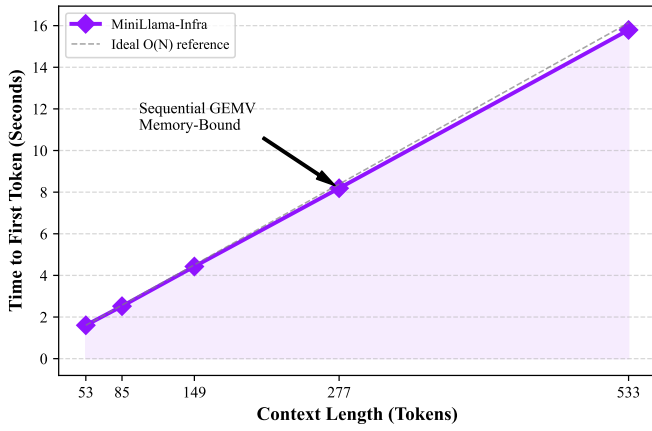


Fig. 4. TTFT scaling with context length. The linear growth confirms the sequential forward pass bottleneck of single-query GEMV-based prefill, producing $O(N^2)$ total compute.

with hardware profiling data that is typically absent from production framework documentation.

VI. CONCLUSION

We presented MiniLlama-Infra, a from-scratch LLM inference engine designed as a micro-architecture ablation framework for edge GPU deployment. Through a five-stage optimization pipeline profiled with NVIDIA Nsight Compute, we demonstrated three key findings: (1) naive INT8 quantization is counterproductive on consumer GPUs due to uncoalesced memory access, with throughput dropping from 58.1 to 19.2 GB/s—a result that underscores the primacy of access pattern optimization over data compression; (2) warp-level cooperative dequantization is the single most critical optimization, alone restoring throughput to 85.6 GB/s and doubling end-to-end decoding speed; (3) once the memory bottleneck is resolved, the system enters a compute-bound regime where further fusion yields diminishing returns. Our hybrid-precision approach achieves 35.2 tok/s at 2054 MB VRAM with a WikiText-2 perplexity of 11.52—0.28 PPL lower than llama.cpp’s global Q8_0—providing a reproducible reference for the hardware-level optimization landscape of edge LLM inference. Future work includes implementing batched GEMM-based prefill and INT8 dot-product instructions (`dp4a`) to further close the throughput gap with production frameworks.

REFERENCES

- [1] H. Touvron et al., “LLaMA: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [2] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [3] G. Gerganov, “llama.cpp,” GitHub repository, 2023. [Online]. Available: <https://github.com/ggerganov/llama.cpp>
- [4] W. Kwon et al., “Efficient memory management for large language model serving with PagedAttention,” in *Proc. ACM SOSP*, 2023.
- [5] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “GPT3.int8(): 8-bit matrix multiplication for transformers at scale,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [6] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *Proc. ICLR*, 2023.
- [7] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” in *Proc. MLSys*, 2024.
- [8] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “SmoothQuant: Accurate and efficient post-training quantization for large language models,” in *Proc. ICML*, 2023.
- [9] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” in *Proc. ICLR*, 2017.